Spring Security – Interview Notes

Spring Boot 3 / Spring Security 6. Highlighted = interview keyword. *Green italics = example.*

A. Basics

1. Spring Security is the Spring framework that handles **authentication** (who are you?) and **authorization** (what are you allowed to do?). It also protects against common attacks such as CSRF and session fixation.

- Add `spring-boot-starter-security` and **every endpoint becomes secured** automatically. You get a default login page, and a default user `user` whose generated password is printed in the logs.

2. Filter Chain: Spring Security works through **servlet filters**, which run **before** a request reaches your controller. Each filter does one job (CORS, CSRF, login, checking a token, checking permissions).

- DelegatingFilterProxy: connects the servlet container (Tomcat) to Spring beans. FilterChainProxy: picks which SecurityFilterChain applies to the request.
- Key filters in order: **CorsFilter** → **CsrfFilter** → **UsernamePasswordAuthenticationFilter** (form login) / **BasicAuthenticationFilter** → *your JWT filter* → **ExceptionTranslationFilter** (turns failures into 401/403) → **AuthorizationFilter** (checks access rules).



*Like airport security: every request passes each checkpoint **before** reaching the controller.*

> **Remember:** Spring Security = a chain of filters in front of your controllers.

B. Authentication Flow (most asked)

3. What happens when a user logs in with a username and password:

- **1.** A filter reads the credentials and creates an **unauthenticated** UsernamePasswordAuthenticationToken.
- **2.** It passes the token to the AuthenticationManager (implementation: **ProviderManager**), which hands it to the right AuthenticationProvider (for username/password: **DaoAuthenticationProvider**).
- **3.** The provider calls UserDetailsService. `loadUserByUsername()` to load the user from the DB as a UserDetails.
- **4.** PasswordEncoder. `matches(raw, hashed)` checks the password.

- **5.** On success, an **authenticated** Authentication object (user + roles) is stored in the SecurityContextHolder. On failure, the response is **401**.



Like an office reception: show ID → manager → check records → verify password → get a badge.

4. Core objects:

- Authentication: represents the user. Holds the **principal** (the user), **credentials** (the password, cleared after login) and **authorities** (roles/permissions).
- SecurityContextHolder: stores the current user's SecurityContext in a **ThreadLocal**, so any code handling that request can read it. *Current user:* `SecurityContextHolder.getContext().getAuthentication()` *, or* `@AuthenticationPrincipal` *in a controller.*
- UserDetails: Spring's view of a user (username, password, authorities, enabled/locked flags). Your User entity can implement it.
- GrantedAuthority: one permission or role, e.g. `ROLE_ADMIN` or `READ_ORDER` .

5. PasswordEncoder: never store plain passwords. BCryptPasswordEncoder hashes the password with a random **salt** and is deliberately slow, which makes brute-force attacks hard. The same password gives a different hash each time; `matches()` still verifies it correctly.

> **Remember:** Filter → Manager → Provider → UserDetailsService → PasswordEncoder → SecurityContextHolder.

C. Configuration

6. SecurityFilterChain bean: in Spring Security 6 you configure security by declaring a bean. The old **WebSecurityConfigurerAdapter was removed**.

```
@Configuration @EnableWebSecurity
public class SecurityConfig {
  @Bean
  SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
      .csrf(csrf -> csrf.disable())                        // stateless REST API
      .authorizeHttpRequests(auth -> auth
          .requestMatchers("/auth/**").permitAll()
          .requestMatchers("/admin/**").hasRole("ADMIN")
          .anyRequest().authenticated())
      .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
      .build();
  }
  @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

7. Where users come from: an **in-memory** user list (`InMemoryUserDetailsManager`) is only for demos. In real apps you write your own **UserDetailsService** that loads users from the DB with JPA:

```
@Service
public class MyUserDetailsService implements UserDetailsService {
  public UserDetails loadUserByUsername(String email) {
    return userRepo.findByEmail(email)
        .orElseThrow(() -> new UsernameNotFoundException("Not found"));
  }
}
```

> **Remember:** Spring Security 6: SecurityFilterChain @Bean + your own UserDetailsService.

D. Authorization

8. URL-based authorization (inside the filter chain): `permitAll()` , `authenticated()` , `hasRole("ADMIN")` , `hasAuthority("READ_ORDER")` . Rules are checked **top to bottom**, so put specific rules first and `anyRequest()` last.

9. Method-level security: add `@EnableMethodSecurity` , then protect individual service or controller methods:

- @PreAuthorize("hasRole('ADMIN')"): checks **before** the method runs (the most used). It can use parameters: `@PreAuthorize("#userId == authentication.principal.id")` .
- @PostAuthorize: checks **after** the method runs, against the returned object. `@Secured("ROLE_ADMIN")` is the older, simpler option.

10. Role vs Authority: both are GrantedAuthority. A **role** is coarse-grained (ADMIN, USER) and stored with the `ROLE_` prefix; `hasRole("ADMIN")` checks for `ROLE_ADMIN` . An **authority** is a fine-grained permission (READ_ORDER, DELETE_USER) checked with `hasAuthority()` .

> **Remember:** URL rules in the chain, @PreAuthorize on methods. hasRole("X") = ROLE_X.

E. Authentication Mechanisms

11. Form Login + Session (stateful): after login, the server creates a session and sends a `JSESSIONID` cookie. Later requests send the cookie and the server looks up the session. Used in traditional server-rendered web apps.

12. HTTP Basic: every request sends `Authorization: Basic base64(user:pass)`. Base64 is an encoding, **not encryption**, so use it only over HTTPS. Fine for internal tools and testing.

13. JWT Authentication (stateless, the standard for REST APIs and microservices):

- **Login**: `POST /auth/login` → `AuthenticationManager.authenticate()` → on success, generate a JWT (using a library such as jjwt) and return it.
- **Every request**: the client sends `Authorization: Bearer <token>`. A custom JwtAuthenticationFilter extends OncePerRequestFilter (runs once per request) does the following:
  extract the token → validate the signature and expiry → get the username → load UserDetails → put an authenticated token in the SecurityContextHolder → continue the chain.
- Register it with `addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)`. Set the session policy to **STATELESS** and **disable CSRF** (no cookies, so there's nothing to forge).
- **JWT structure**: `header.payload.signature`. The payload is only Base64-encoded, so anyone can read it: **never put passwords in a JWT**. The signature proves it hasn't been tampered with.



Session = coat-check ticket (the server keeps records). JWT = a festival wristband (checked at the door, no records needed).

14. Access + Refresh Token: the access token is short-lived (~15 min) and sent with every API call. The refresh token is long-lived (days), stored in the DB and an HttpOnly cookie, and sent only to `/auth/refresh` to get a new access token. To log out, revoke the refresh token. With **rotation**, every refresh also issues a new refresh token.



Access token = a 15-minute day pass. Refresh token = a membership card used to get a new pass.

15. OAuth2 / OIDC:

- **OAuth2** = *authorization*: a user lets an app access their data without sharing their password. **OIDC (OpenID Connect)** = a layer on OAuth2 that adds *authentication*, giving an **ID token** with the user's identity.
- oauth2Login(): your app acts as the **client**. *"Login with Google/GitHub". Configure the client-id and secret in application.yml.*
- oauth2ResourceServer().jwt(): your API is a **resource server** that only **validates** JWTs issued by an auth server such as **Keycloak, Okta or Auth0**, so you don't write a JWT filter yourself. The common microservices setup.

> **Remember:** Session = stateful cookie. JWT = stateless signed token checked by OncePerRequestFilter.

F. Protection & Error Handling

16. CSRF (Cross-Site Request Forgery): you're logged into your bank, which has stored a cookie in your browser, and you visit an evil site. That site makes your browser send `POST bank.com/transfer`, and the browser **automatically attaches your cookie**, so the request succeeds.

- Fix: the server requires a secret **CSRF token** on every form or POST request that the evil site can't know. It's **enabled by default**. **Disable it only for stateless JWT APIs**, which don't rely on cookies.



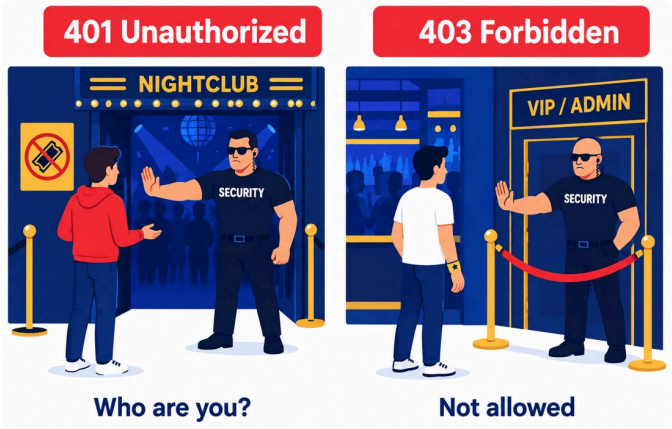
The browser sends the bank cookie automatically. A CSRF token is the one thing evil.com can't forge.

17. CORS (Cross-Origin Resource Sharing): browsers block JavaScript on `localhost:3000` (React) from calling an API on `localhost:8080` unless the API allows that origin. Configure a `CorsConfigurationSource` bean

(allowed origins, methods, headers) and enable `http.cors()`. The browser first sends an **OPTIONS preflight** request to check permission.

18. 401 vs 403:

- **401 Unauthorized** = not authenticated (no token or a bad token). Customise it with an AuthenticationEntryPoint.
- **403 Forbidden** = logged in but not allowed. *A USER calling an /admin API.* Customise it with an AccessDeniedHandler.



401 = no ticket at the door. 403 = inside the club, but not allowed into the VIP room.

19. Other built-in protections: Session fixation protection (a new session ID is issued after login), security headers (X-Frame-Options against clickjacking, HSTS to force HTTPS), and logout handling (`/logout` clears the session and the context).

> **Remember:** CSRF only matters with cookies. CORS is enforced by the browser. 401 = who are you, 403 = not allowed.

G. Quick Revision

Question	Answer in one line
How does Spring Security work?	A chain of servlet filters that runs before the DispatcherServlet
Who checks the password?	DaoAuthenticationProvider, using UserDetailsService + PasswordEncoder
Where is the logged-in user stored?	SecurityContextHolder (ThreadLocal)
How do you add JWT?	OncePerRequestFilter + addFilterBefore + STATELESS + disable CSRF
Replacement for WebSecurityConfigurerAdapter?	A SecurityFilterChain @Bean
hasRole vs hasAuthority?	hasRole adds the ROLE_ prefix automatically
@PreAuthorize vs @PostAuthorize?	Checks before vs after the method runs
Why disable CSRF for JWT?	No cookies, so the browser doesn't attach credentials automatically
401 vs 403?	Not logged in vs logged in but not allowed
Session vs JWT?	Server stores state (stateful) vs self-contained signed token (stateless)